

Inferencia reproducible: del CSV a la predicción

Asignatura: Operación de Modelos

Profesores: Andrés Jiménez y Sergio Sillero

MUIAAp 2026/2027



CÓDIGO

Material y Contenido

1. Conceptos Fundamentales

Diferenciar entrenamiento, modelo e inferencia.
Entender qué ocurre cuando recibimos datos nuevos.
Identificar errores habituales en producción.

2. Contratos de datos

Definir un contrato de entrada.
Validar tipos, rangos y columnas permitidas.
Diseñar un contrato de salida trazable.

3. Preprocesado e inferencia

Feature Engineering
Carga de Joblib
Predict y guardado

4. Preparación del taller

Seguir una muestra desde el CSV hasta la predicción.
Interpretar el resultado generado.
Entender la estructura del módulo que construirán en la clase 2.

01.

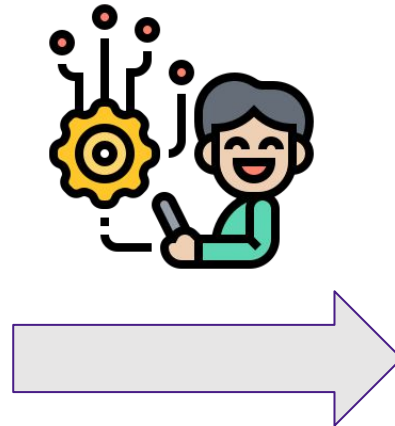
Conceptos Fundamentales



Punto de Partida

Nos entregan:

- Un modelo ya entrenado en formato `.joblib`.
- Un CSV con nuevas muestras de vino.
- Ninguna etiqueta de calidad para esas muestras.

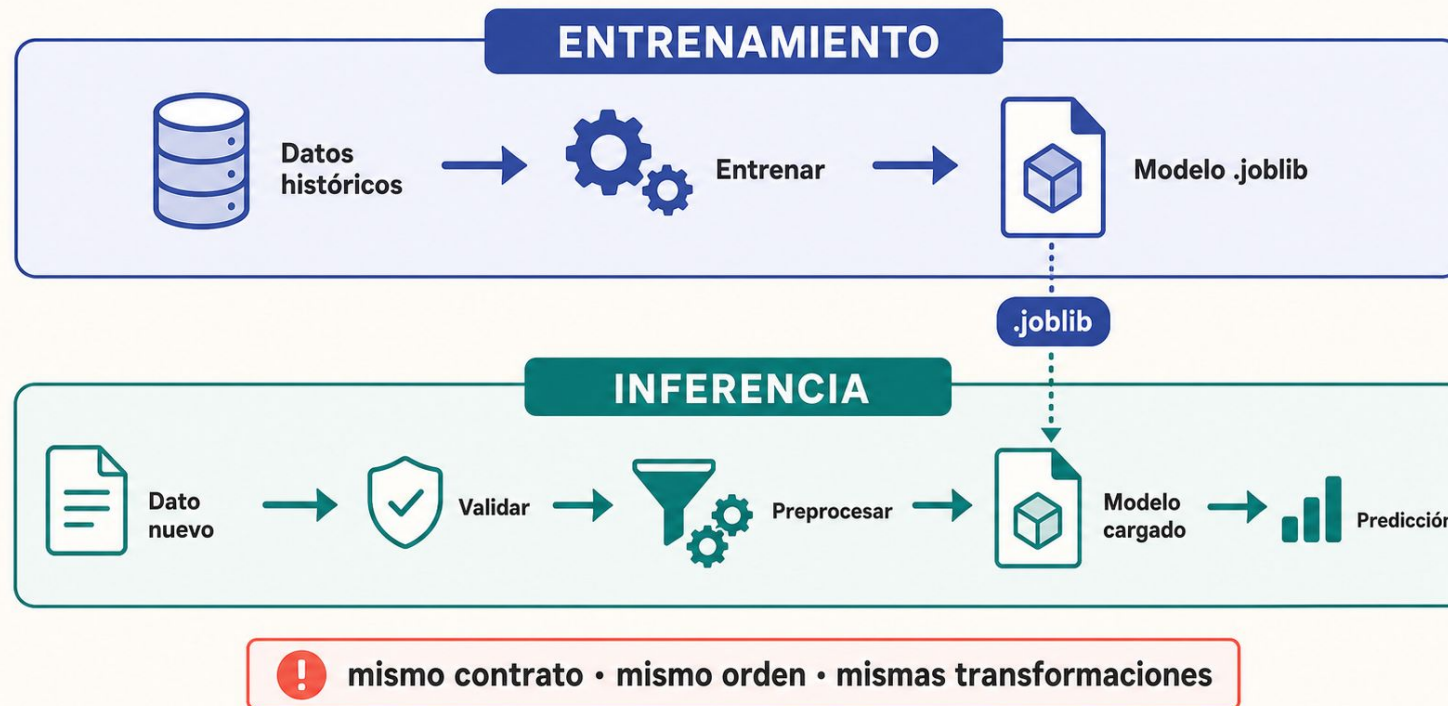


Nuestro trabajo:

Convertir datos nuevos
en predicciones fiables y
trazables.

¿Qué es un contrato de datos?

Entrenamiento $\neq$ inferencia



¿Por qué puede fallar un modelo que “funciona”?

¡Que el código se ejecute no significa que la predicción sea correcta!



El modelo puede producir una predicción aunque:

- Falte una característica.
- Aparezca una columna desconocida.
- Cambie el orden de las variables.
- Se utilicen unidades diferentes.
- El preprocesado sea distinto al del entrenamiento.
- Que las hipótesis vistas en el entrenamiento sean distintas a la realidad del problema

02.

Contratos de datos



Ejemplo de contrato de Datos



Bringing schema and sanity to your data



```
from pydantic import BaseModel, ConfigDict, Field, field_validator

class WineQualityRequest(BaseModel):
    model_config = ConfigDict(extra="forbid")

    sample_id: str = Field(min_length=1)

    fixed_acidity: float = Field(ge=0, le=20)
    volatile_acidity: float = Field(ge=0, le=2)
    citric_acid: float = Field(ge=0, le=2)
    residual_sugar: float = Field(ge=0, le=20)
    chlorides: float = Field(ge=0, le=1)
    free_sulfur_dioxide: float = Field(ge=0, le=100)
    total_sulfur_dioxide: float = Field(ge=0, le=300)
    density: float = Field(ge=0.9, le=1.1)
    ph: float = Field(ge=0, le=14)
    sulphates: float = Field(ge=0, le=2)
    alcohol: float = Field(ge=0, le=25)

    @field_validator("sample_id")
    @classmethod
    def sample_id_not_blank(cls, value: str) -> str:
        value = value.strip()

        if not value:
            raise ValueError("sample_id no puede estar vacío")

        return value
```

```
@dataclass(frozen=True, slots=True)
class WineQualityRequest:
    sample_id: str

    fixed_acidity: float
    volatile_acidity: float
    citric_acid: float
    residual_sugar: float
    chlorides: float
    free_sulfur_dioxide: float
    total_sulfur_dioxide: float
    density: float
    ph: float
    sulphates: float
    alcohol: float

    def __post_init__(self) -> None:
        if not isinstance(self.sample_id, str):
            raise TypeError("sample_id debe ser texto")

        if not self.sample_id.strip():
            raise ValueError("sample_id no puede estar vacío")

        ranges = (
            ("fixed_acidity", self.fixed_acidity, 0, 20),
            ("volatile_acidity", self.volatile_acidity, 0, 2),
            ("citric_acid", self.citric_acid, 0, 2),
            ("residual_sugar", self.residual_sugar, 0, 20),
            ("chlorides", self.chlorides, 0, 1),
            ("free_sulfur_dioxide", self.free_sulfur_dioxide, 0, 100),
            ("total_sulfur_dioxide", self.total_sulfur_dioxide, 0, 300),
            ("density", self.density, 0.9, 1.1),
            ("ph", self.ph, 0, 14),
            ("sulphates", self.sulphates, 0, 2),
            ("alcohol", self.alcohol, 0, 25),
        )

        for name, value, minimum, maximum in ranges:
            check_range(name, value, minimum, maximum)
```


Contrato de datos

El contrato de entrada

Un contrato define:

- Qué campos se aceptan.
- Qué tipo tiene cada campo.
- Qué valores son válidos.
- Qué campos están prohibidos.
- Qué información no debe llegar al modelo.

En nuestro caso:

- `sample_id` identifica la muestra.
- Las otras 11 columnas son características numéricas.
- No se permiten columnas adicionales.

```
from pydantic import BaseModel, ConfigDict, Field, field_validator

class WineQualityRequest(BaseModel):
    model_config = ConfigDict(extra="forbid")

    sample_id: str = Field(min_length=1)

    fixed_acidity: float = Field(ge=0, le=20)
    volatile_acidity: float = Field(ge=0, le=2)
    citric_acid: float = Field(ge=0, le=2)
    residual_sugar: float = Field(ge=0, le=20)
    chlorides: float = Field(ge=0, le=1)
    free_sulfur_dioxide: float = Field(ge=0, le=100)
    total_sulfur_dioxide: float = Field(ge=0, le=300)
    density: float = Field(ge=0.9, le=1.1)
    ph: float = Field(ge=0, le=14)
    sulphates: float = Field(ge=0, le=2)
    alcohol: float = Field(ge=0, le=25)

    @field_validator("sample_id")
    @classmethod
    def sample_id_not_blank(cls, value: str) -> str:
        value = value.strip()

        if not value:
            raise ValueError("sample_id no puede estar vacío")

        return value
```

Práctica 1: leer una fila del CSV



Bringing schema and sanity to your data

Abrimos `inference_samples.csv`.

Preguntas para las parejas:

1. ¿Qué campo identifica la petición?
2. ¿Qué campos consume el modelo?
3. ¿Qué columna añadirías para comprobar que el contrato rechaza campos extra?
4. ¿Qué valores considerarías inválidos?

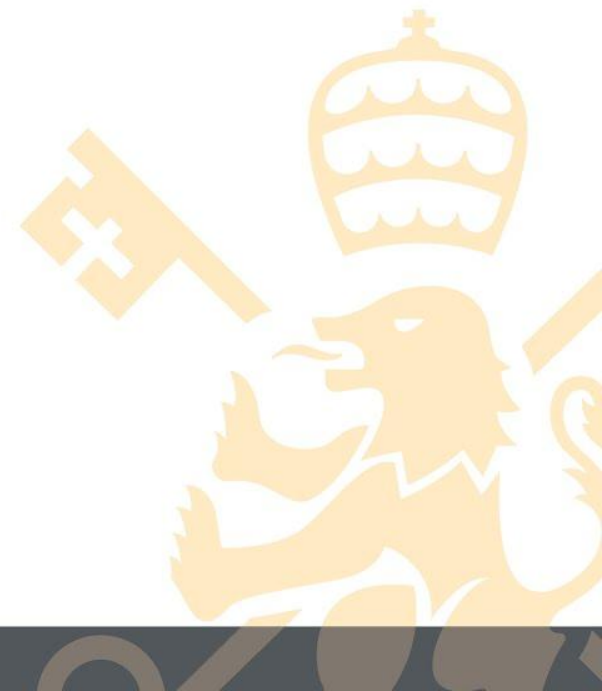
Código

¿Qué features tiene el modelo?

Grupo	Variables
Acidez	<code>fixed_acidity</code> , <code>volatile_acidity</code> , <code>citric_acid</code> , <code>ph</code>
Composición	<code>residual_sugar</code> , <code>chlorides</code> , <code>density</code>
Sulfatos y azufre	<code>sulphates</code> , <code>free_sulfur_dioxide</code> , <code>total_sulfur_dioxide</code>
Alcohol	<code>alcohol</code>

`sample_id` acompaña a la muestra, pero no entra en el vector del modelo.

Variable **Target**: `Quality` (valores del 0 al 10)



Del CSV al objeto validado

Fila CSV



WineQualityRequestModel



Tipos y rangos comprobados



Datos listos para preprocesar
(feature engineering)

Ejemplo conceptual:

"alcohol": "9.4"



alcohol: float = 9.4

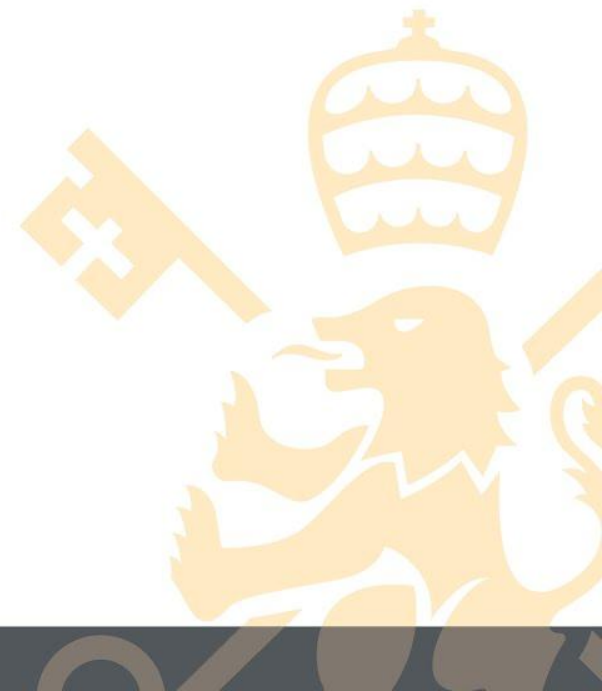
La **validación debe ocurrir antes** de la inferencia.

¿Qué significa preprocesar?

Preprocesar significa transformar una petición válida en exactamente el formato que espera el modelo:

- Convertir tipos.
- Adaptar nombres.
- Mantener el orden.
- Aplicar las mismas transformaciones del entrenamiento.
- Registrar una versión del preprocesado.

En este ejercicio no entrenamos ni reescalamos de nuevo el modelo.

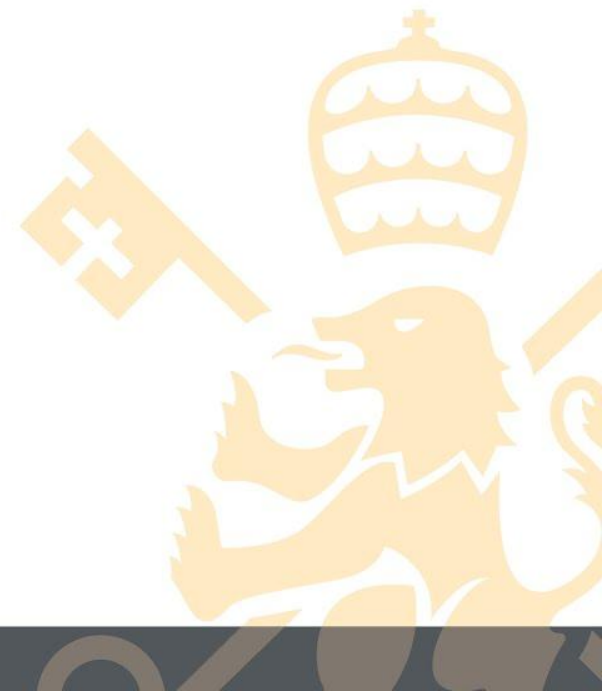


El orden de las features importa

El modelo espera un array con 11 valores con un orden estricto:

```
[  
  fixed_acidity,  
  volatile_acidity,  
  citric_acid,  
  residual_sugar,  
  chlorides,  
  free_sulfur_dioxide,  
  total_sulfur_dioxide,  
  density,  
  ph,  
  sulphates,  
  alcohol  
]
```

Si intercambiamos `alcohol` y `density`, el modelo recibe valores, pero en posiciones equivocadas. (no es la primera vez que ocurre... sobre todo en notebooks donde no validas)



¿Sabías que el binario del modelo guarda metadatos relevantes?

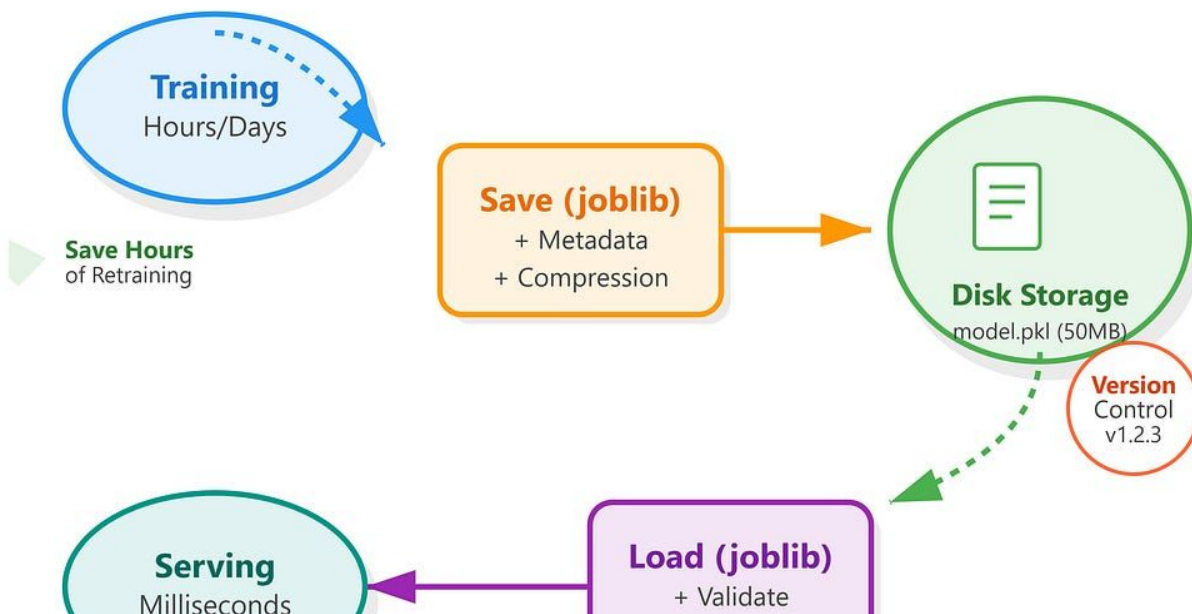
El artefacto incluye:

- El estimador entrenado.
- Los nombres y orden de características.
- La versión del modelo.
- La versión de la librería con la que fué entrenado

Antes de inferir debemos comprobar:

```
feature_names del artefacto  
==  
feature_names del contrato
```

Model Persistence: Train Once, Serve Forever



Contrato de salida (Predict)

Cada predicción devuelve:

Campo	Significado
<code>sample_id</code>	Identificador original
<code>quality_band</code>	Categoría predicha
<code>confidence</code>	Confianza del clasificador
<code>model_version</code>	Versión del modelo
<code>preprocessing_version</code>	Versión del preprocesado



La salida debe poder interpretarse y auditarse después.

¿Qué responsabilidad tendría cada archivo?

CSV
↓
contracts.py
↓
preprocess.py
↓
inference.py
↓
predict_file.py
↓
predictions.csv

```
starter/  
├── assets/  
├── src/  
│   ├── model_inference/  
│   │   ├── contracts.py  
│   │   ├── preprocess.py  
│   │   ├── inference.py  
│   │   └── predict_file.py  
└── tests/
```

03.

Preprocesado e Inferencia



¿Qué responsabilidad tendría cada archivo?



```
starter/  
├── assets/  
├── src/  
│   ├── model_inference/  
│   │   ├── contracts.py  
│   │   ├── preprocss.py  
│   │   ├── inference.py  
│   │   └── predict_file.py  
└── tests/
```

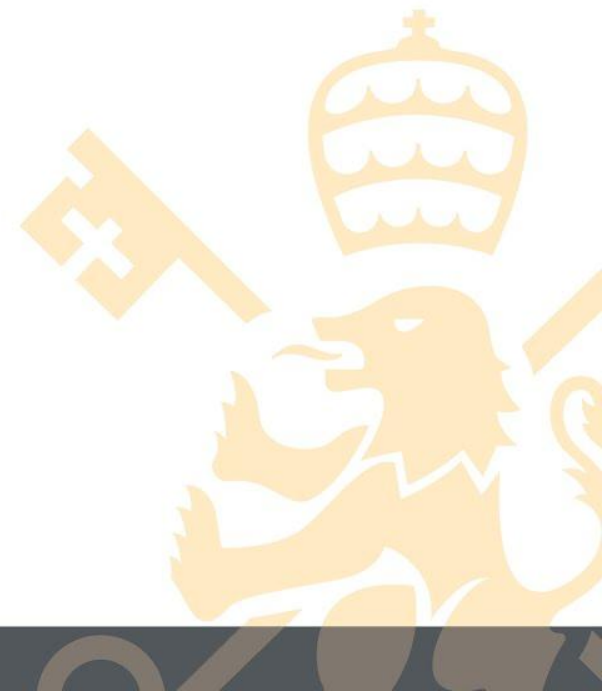
¿Qué hace cada script del modulo?

CSV
↓
contracts.py
↓
preprocess.py
↓
inference.py
↓
predict_file.py
↓
predictions.csv

Responsabilidad de cada parte:

- `contracts.py`: validar.
- `preprocess.py`: preparar.
- `inference.py`: cargar y predecir.
- `predict_file.py`: coordinar el flujo.

model_version: wine-quality-rf-v1
preprocessing_version: wine-red-features-v1



Ejecutemos un script de inferencia

```
uv run python -m model_inference.predict_file \  
  --input assets/03-wine-quality/inference_samples.csv \  
  --output .tmp/wine_predictions.csv
```

El script:

1. Lee el CSV.
2. Valida las filas.
3. Preprocesa las características.
4. Carga el modelo.
5. Escribe las predicciones.



COMILLAS
UNIVERSIDAD PONTIFICIA

ICAI

ICADE

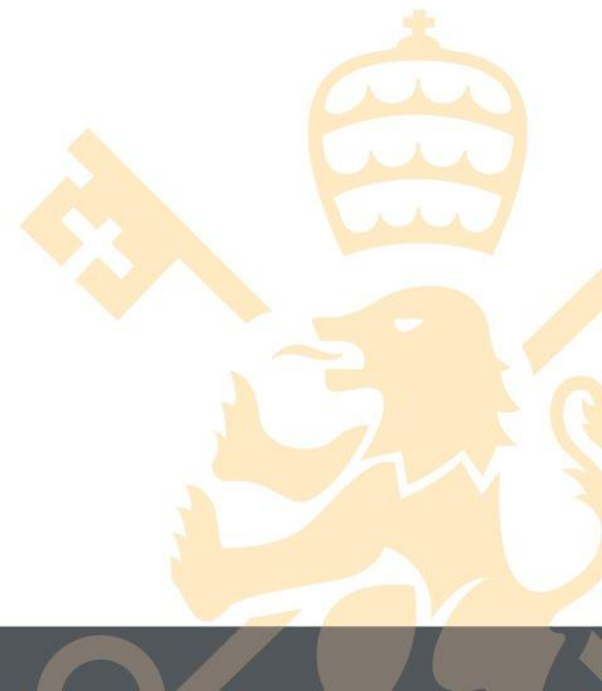
CIHS

Ejecutemos un script de inferencia

red-001
↓
WineQualityRequest
↓
WineFeatures
↓
vector ordenado de 11 valores
↓
RandomForestClassifier
↓
needs_review · 0.77

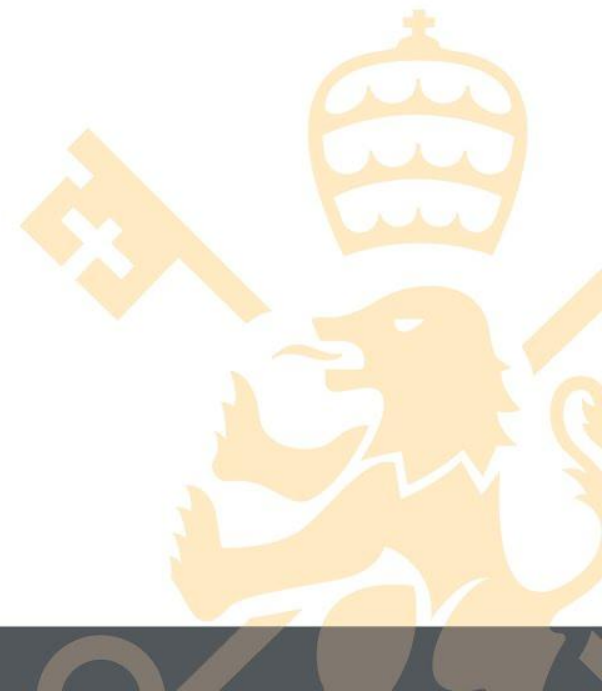
Pregunta:

¿En qué punto se valida cada parte?



Resultado

sample_id	quality_band	confidence
red-001	needs_review	0.77
red-002	needs_review	0.80
red-003	acceptable	0.58
red-004	needs_review	0.66
red-005	needs_review	0.47



Provocar un error en el contrato

Añadir al CSV:

`unexpected_field = "not allowed"`

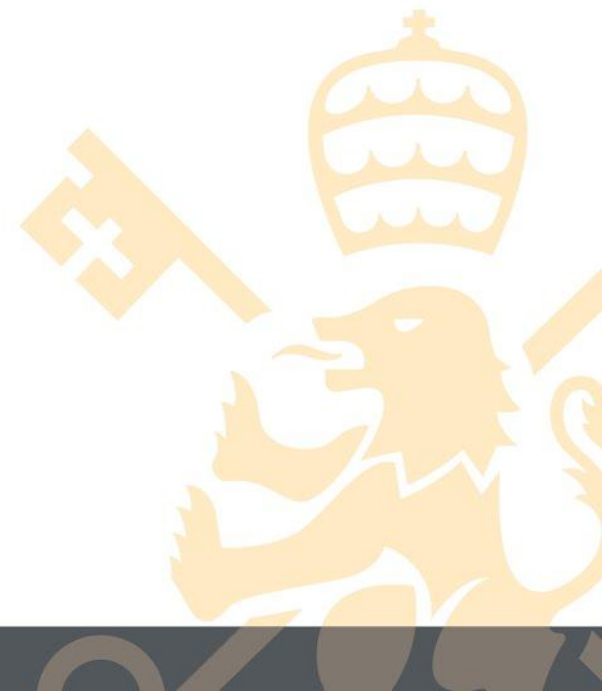
Resultado esperado:

Error de validación:

el campo `unexpected_field` no está permitido

El script debe:

- Fallar con un mensaje claro.
- No llamar al modelo.
- No crear un CSV parcial.



Los tests como especificación

Los tests describen cinco comportamientos:

1. Una fila válida produce una predicción.
2. Una columna desconocida se rechaza.
3. Una fila posterior inválida no deja salida parcial.
4. Un `sample_id` vacío se rechaza.
5. Un artefacto con características incompatibles se rechaza.



COMILLAS
UNIVERSIDAD PONTIFICIA

ICAI

ICADE

CIHS

comillas.edu

